



UNIVERSITY OF LAY ADVENTISTS OF KIGALI — UNILAK

Laundry Management System

An Oracle Database & PL/SQL Project

Course: Database Programming

Student Name: G. Uriah Summerville, Jr.

Registration Number: 28873/2025

Submission Date: Thursday, July 30, 2026

1

Introduction

Phase I — Project Overview

- ◆ The Laundry Management System (LMS) is an Oracle-based database application designed to digitize the daily operations of a laundry business from order intake to payment and pickup.
- ◆ It replaces manual, paper-based registers currently used by small and medium laundry shops with a centralized, reliable, and query-able Oracle database.
- ◆ The project is built in progressive phases: business process modeling, logical database design, physical implementation, PL/SQL programming, and advanced security/auditing features.
- ◆ Technologies used: Oracle Database, SQL*Plus / SQL Developer, PL/SQL (procedures, functions, packages, triggers).

Project Roadmap

- I. Introduction
- II. Problem Statement
- III. Business Process Modeling
- IV. Logical Database Design
- V. Database Creation
- VI. Table Creation & Implementation
- VII. PL/SQL Programming
- VIII. Advanced DB Programming
- IX. Conclusion & Reference

2

Problem Statement

Phase II — Real-Life Context & Objectives

Context of Use: “There are many Laundry around that currently records customer orders, service prices, and payments by hand in notebooks. This causes lost order slips, incorrect billing, no visibility into which attendant is handling which order, and no historical record for decision-making.

Target Users

- ◆ Shop Manager — oversees operations & reports
- ◆ Front-Desk Officer — registers orders & payments
- ◆ Laundry Attendants — process & wash items
- ◆ Customers — receive accurate, fast service

Project Objectives

- ◆ Centralize customer & order data in Oracle
- ◆ Track order status from drop-off to pickup
- ◆ Automate pricing & billing calculations
- ◆ Enforce data integrity and secure access
- ◆ Provide audit trail of all data changes

Expected Benefits

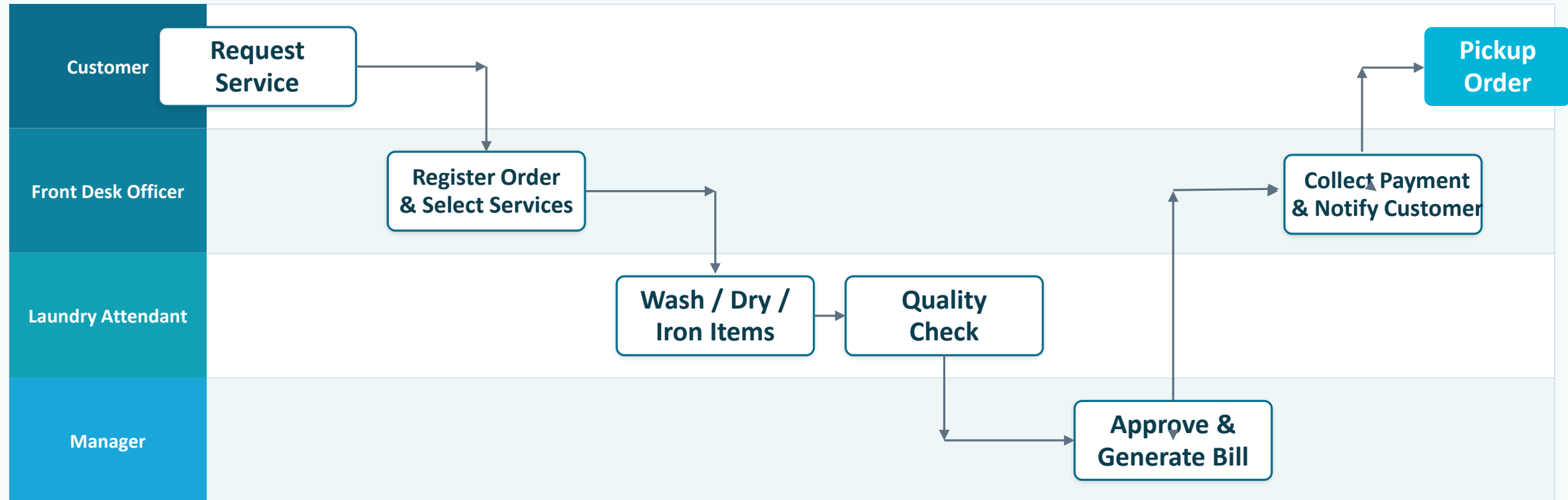
- ◆ Faster, error-free order processing
- ◆ Real-time tracking of laundry status
- ◆ Accurate, transparent billing for customers
- ◆ Reliable records for audits & performance
- ◆ Reduced fraud via activity logging

3

Business Process Modeling

Phase III — Scope, Actors & Swimlane Workflow

System Scope: order intake → service selection → washing → quality check → payment → pickup. **Actors:** Customer, Front-Desk Officer, Laundry Attendant, Manager (approvals & reports).

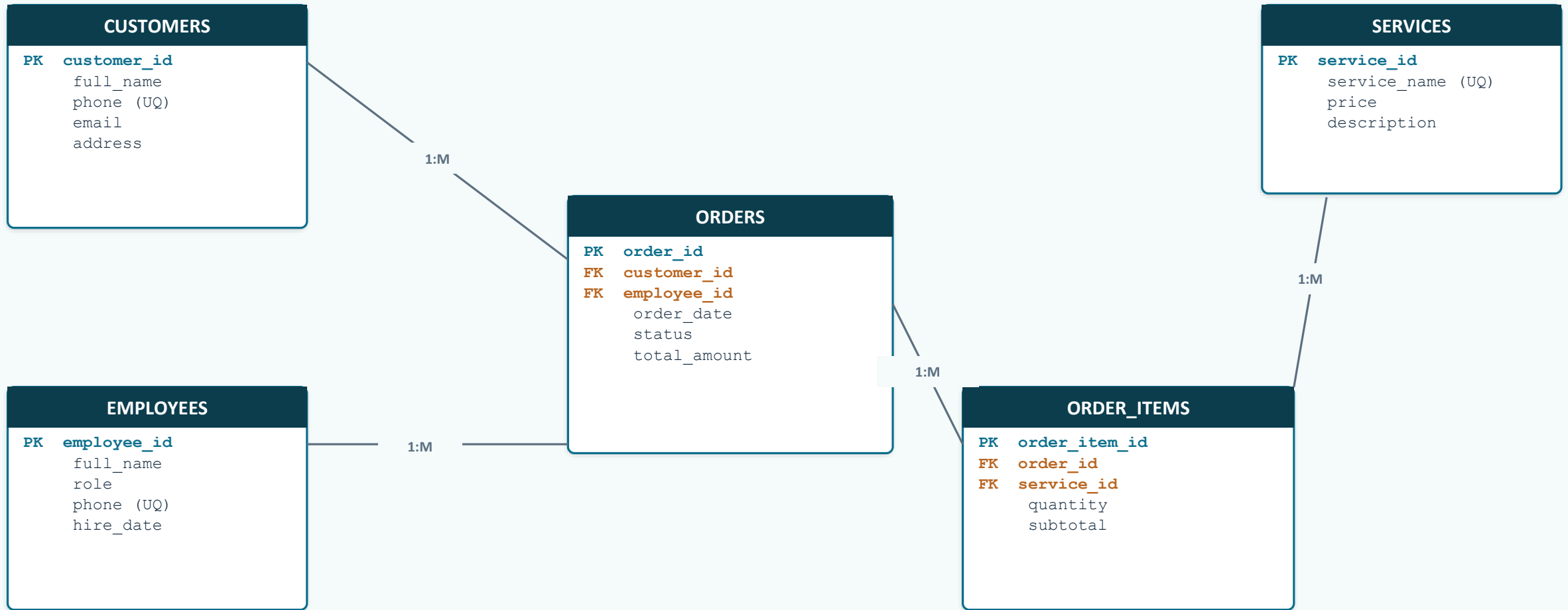


Workflow (BPMN-style swimlane, start→end): a customer requests service; the front-desk officer registers the order in ORDERS/ORDER_ITEMS; attendants wash and quality-check items; the manager approves the bill; the front desk collects payment and the customer picks up the order.

4

Logical Database Design

Phase IV — ER Diagram, Keys, Relationships & 3NF (5 tables)



Normalized to 3NF: atomic attributes (1NF); single-column surrogate PKs → no partial dependency (2NF); no transitive dependencies — `total_amount` is a maintained/derived value, not a functional dependency of another non-key attribute (3NF).

```
-- Create dedicated application schema/user
CREATE USER laundry_admin
  IDENTIFIED BY "Laundry#2026"
  DEFAULT TABLESPACE users
  TEMPORARY TABLESPACE temp
  QUOTA UNLIMITED ON users;

-- Assign least-privilege access
GRANT CREATE SESSION, CREATE TABLE,
      CREATE VIEW, CREATE PROCEDURE,
      CREATE TRIGGER, CREATE SEQUENCE
  TO laundry_admin;

-- Configure access (SQL*Plus)
CONNECT laundry_admin/Laundry#2026@orclpdb
```

Naming Convention

- ◆ Tables: plural UPPER_SNAKE_CASE — e.g. CUSTOMERS, ORDERS
- ◆ Primary keys: <table_singular>_id — e.g. customer_id
- ◆ Foreign keys: fk_<child>_<parent> — e.g. fk_orders_customers
- ◆ Constraints: pk_/fk_/uq_/ck_ prefix by type
- ◆ Triggers: trg_<table>_<purpose>

Access Configuration

- ◆ Application role LMS_APP created; end-user accounts granted the role, not direct object privileges
- ◆ Front-desk & attendant accounts restricted to DML only (no DDL) on business tables
- ◆ Manager account additionally granted SELECT on AUDIT_LOG for oversight

```
CREATE TABLE customers (  
    customer_id NUMBER GENERATED ALWAYS AS IDENTITY  
        PRIMARY KEY,  
    full_name  VARCHAR2(80)  NOT NULL,  
    phone      VARCHAR2(15)  NOT NULL UNIQUE,  
    email      VARCHAR2(100),  
    address    VARCHAR2(150));  
  
CREATE TABLE employees (  
    employee_id NUMBER GENERATED ALWAYS AS IDENTITY  
        PRIMARY KEY,  
    full_name  VARCHAR2(80) NOT NULL,  
    role       VARCHAR2(20) NOT NULL  
        CHECK (role IN ('MANAGER','ATTENDANT',  
                        'WASHER','CASHIER')),  
    phone      VARCHAR2(15) UNIQUE,  
    hire_date  DATE DEFAULT SYSDATE NOT NULL);  
  
CREATE TABLE services (  
    service_id  NUMBER GENERATED ALWAYS AS IDENTITY  
        PRIMARY KEY,  
    service_name VARCHAR2(50) NOT NULL UNIQUE,  
    price       NUMBER(8,2)  NOT NULL CHECK (price > 0),  
    description  VARCHAR2(200));
```

```
CREATE TABLE orders (  
    order_id  NUMBER GENERATED ALWAYS AS IDENTITY  
        PRIMARY KEY,  
    customer_id NUMBER NOT NULL,  
    employee_id NUMBER NOT NULL,  
    order_date DATE DEFAULT SYSDATE NOT NULL,  
    status     VARCHAR2(20) DEFAULT 'PENDING'  
        CHECK (status IN ('PENDING','WASHING','READY',  
                          'COMPLETED','CANCELLED')),  
    total_amount NUMBER(10,2) DEFAULT 0  
        CHECK (total_amount >= 0),  
    CONSTRAINT fk_orders_customers FOREIGN KEY(customer_id)  
        REFERENCES customers(customer_id),  
    CONSTRAINT fk_orders_employees FOREIGN KEY(employee_id)  
        REFERENCES employees(employee_id));  
  
CREATE TABLE order_items (  
    order_item_id NUMBER GENERATED ALWAYS AS IDENTITY  
        PRIMARY KEY,  
    order_id  NUMBER NOT NULL,  
    service_id NUMBER NOT NULL,  
    quantity  NUMBER(4) NOT NULL CHECK (quantity > 0),  
    subtotal  NUMBER(10,2) NOT NULL,  
    CONSTRAINT fk_items_orders FOREIGN KEY(order_id)  
        REFERENCES orders(order_id) ON DELETE CASCADE,  
    CONSTRAINT fk_items_services FOREIGN KEY(service_id)  
        REFERENCES services(service_id));
```

```
-- Sample data & integrity (Phase VI)
INSERT INTO customers (full_name, phone, email)
VALUES ('Aline Uwase','0788123456','aline@mail.com');
INSERT INTO employees (full_name, role)
VALUES ('Eric Niyonzima','ATTENDANT');
INSERT INTO services (service_name, price)
VALUES ('Wash & Fold', 1500);
INSERT INTO orders (customer_id, employee_id)
VALUES (1, 1);
INSERT INTO order_items (order_id, service_id, quantity,
    subtotal) VALUES (1, 1, 3, 4500);
COMMIT;

-- Package specification
CREATE OR REPLACE PACKAGE pkg_laundry AS
    PROCEDURE add_order_item(p_order_id NUMBER,
        p_service_id NUMBER, p_qty NUMBER);
    FUNCTION get_order_total(p_order_id NUMBER)
        RETURN NUMBER;
END pkg_laundry;
/
```

```
-- Package body: procedure, function,
-- cursor loop, exceptions, COMMIT/ROLLBACK
CREATE OR REPLACE PACKAGE BODY pkg_laundry AS
    FUNCTION get_order_total(p_order_id NUMBER)
        RETURN NUMBER IS
        v_total NUMBER := 0;
    BEGIN
        SELECT NVL(SUM(subtotal),0) INTO v_total
            FROM order_items WHERE order_id=p_order_id;
        RETURN v_total;
    END;

    PROCEDURE add_order_item(p_order_id NUMBER,
        p_service_id NUMBER, p_qty NUMBER) IS
        v_price services.price%TYPE;
        e_bad_qty EXCEPTION;
    BEGIN
        IF p_qty <= 0 THEN RAISE e_bad_qty; END IF;
        SELECT price INTO v_price FROM services
            WHERE service_id = p_service_id;
        INSERT INTO order_items (order_id, service_id,
            quantity, subtotal)
            VALUES (p_order_id, p_service_id, p_qty,
                v_price*p_qty);

        COMMIT;
    EXCEPTION
        WHEN e_bad_qty THEN ROLLBACK;
        RAISE_APPLICATION_ERROR(-20010, 'Qty must be >0');
        WHEN NO_DATA_FOUND THEN ROLLBACK;
        RAISE_APPLICATION_ERROR(-20011, 'Invalid service');
    END;
END pkg_laundry;
/
```



```
-- Simple trigger: auto-maintain order total
CREATE OR REPLACE TRIGGER trg_items_total
AFTER INSERT OR UPDATE OR DELETE ON order_items
FOR EACH ROW
BEGIN
    UPDATE orders SET total_amount = (
        SELECT NVL(SUM(subtotal),0) FROM order_items
        WHERE order_id = NVL(:NEW.order_id,:OLD.order_id))
    WHERE order_id = NVL(:NEW.order_id,:OLD.order_id);
END;
/
```

```
-- Compound trigger: audit + activity tracking
CREATE OR REPLACE TRIGGER trg_orders_audit
FOR INSERT OR UPDATE OR DELETE ON orders
COMPOUND TRIGGER
    TYPE t_ids IS TABLE OF NUMBER; g_ids t_ids:=t_ids();
    AFTER EACH ROW IS BEGIN
        g_ids.EXTEND; g_ids(g_ids.COUNT):=
            NVL(:NEW.order_id,:OLD.order_id);
    END AFTER EACH ROW;
    AFTER STATEMENT IS BEGIN
        FOR i IN 1..g_ids.COUNT LOOP
            INSERT INTO audit_log(table_name,action,
                record_id,changed_by) VALUES('ORDERS',
                CASE WHEN UPDATING THEN 'UPDATE'
                WHEN DELETING THEN 'DELETE' ELSE 'INSERT'
            END, g_ids(i),
            SYS_CONTEXT('USERENV','SESSION_USER'));
        END LOOP;
    END AFTER STATEMENT;
END trg_orders_audit;
/
```

```
-- Reference & audit tables (support Phase VIII)
CREATE TABLE public_holidays (
    holiday_date DATE PRIMARY KEY,
    description VARCHAR2(100));

CREATE TABLE audit_log (
    audit_id NUMBER GENERATED ALWAYS AS IDENTITY
        PRIMARY KEY,
    table_name VARCHAR2(30),
    action VARCHAR2(10),
    record_id NUMBER,
    changed_by VARCHAR2(30),
    changed_on DATE DEFAULT SYSDATE);
```

```
-- Security: block writes on weekends & holidays
CREATE OR REPLACE TRIGGER trg_orders_restrict
BEFORE INSERT OR UPDATE OR DELETE ON orders
FOR EACH ROW
DECLARE v_hol NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_hol FROM public_holidays
    WHERE holiday_date = TRUNC(SYSDATE);
    IF TO_CHAR(SYSDATE,'DY') IN ('SAT','SUN')
    OR v_hol > 0 THEN
        RAISE_APPLICATION_ERROR(-20001,
            'Writes allowed Mon-Fri only (business days).');
    END IF;
END;
/
```

Conclusion

- ✦ The Laundry Management System demonstrates a complete Oracle database lifecycle — from business process modeling to secure, audited, production-style implementation.
- ✦ A compact, normalized 5-table schema (CUSTOMERS, EMPLOYEES, SERVICES, ORDERS, ORDER_ITEMS) supports the full order-to-payment workflow while remaining easy to maintain.
- ✦ PL/SQL packages, triggers, and constraints enforce business rules automatically, reducing manual errors seen in the original paper-based process.
- ✦ Compound triggers and the AUDIT_LOG / PUBLIC_HOLIDAYS reference tables add accountability and enforce weekday-only business operations.
- ✦ The system is realistic, scalable, and ready for extension (e.g. loyalty points, SMS notifications, multi-branch support).

References

- ✦ Oracle Corp., “Oracle Database SQL Language Reference,” Oracle Help Center, 2024.
- ✦ Oracle Corp., “PL/SQL Language Reference,” Oracle Help Center, 2024.
- ✦ Elmasri, R. & Navathe, S., Fundamentals of Database Systems, 7th ed., Pearson, 2016.
- ✦ Object Management Group, “Business Process Model and Notation (BPMN) 2.0,” OMG, 2011.
- ✦ UNILAK, Database Programming Course Notes, 2026.